

Informe 2

Elaborado por:

Alejandro Zambrano (17-10684), Ángel Rodríguez (15-11669), Francisco Márquez (12-11163), Kevin Briceño (15-11661), Sergio Carrillo (14-11315).

Fecha: 22/02/2026

Introducción

El problema de satisfacibilidad booleana (SAT) es el problema de saber si una fórmula en forma normal conjuntiva tiene una asignación de variables que la haga verdadera. En casos más complejos o directamente en casos de contradicciones, no se puede satisfacer la fórmula pero se busca minimizar las cláusulas que están insatisfechas: este es el problema MaxSAT. Tal problema cuenta con varias especializaciones sobre las cláusulas (que pueden tener ponderación o algún tipo de condición de obligatoriedad). En este estudio se trabajó con el problema MaxSAT puro.

Para el estudio de este problema se usó el *benchmark Causal Discovery* el cual es una transcripción de problemas de estadística para encontrar relaciones de causalidad entre sucesos correlacionados. Este *benchmark* explota al algoritmo de un solucionador por dar lugar a cláusulas densas (con muchas variables) por lo que es impráctico plantear un solucionador exacto en este estudio y se usará uno disponible en la red para comparar la solución heurística y metaheurísticas de búsqueda local, búsqueda voraz aleatorizada, recocido simulado y algoritmo genético propuestas.

Descripción del problema

El problema de satisfacibilidad booleana (SAT) es el problema de saber si dada una fórmula booleana en forma normal conjuntiva (conjunción de disyunciones), existe alguna asignación de variables tal que la fórmula sea verdadera (se satisfaga). El problema MaxSAT es un problema de optimización donde dada una fórmula SAT, que muy probablemente es insatisfacible (una contradicción) o donde muy pocas asignaciones de variables la satisfacen, se quiere una asignación de variables tal que se minimice la cantidad de cláusulas insatisfechas o, equivalentemente, se maximice las cláusulas satisfechas. [1]

En la literatura, existen varias especializaciones de MaxSAT [1]:

- MaxSAT ponderado: existen cláusulas que tienen mayor preferencia para ser satisfechas (tienen mayor peso o ponderación).
- MaxSAT parcial: existen cláusulas que obligatoriamente deben satisfacerse (*hard*) y otras que se permite que no se satisfagan (*soft*).
- MaxSAT ponderado parcial: Una combinación de los casos anteriores.

En este estudio se trabajará con el problema simple MAXSAT donde puede decirse que todas las cláusulas son *soft* y con la misma ponderación con el enfoque de minimización de cláusulas insatisfechas.

Benchmark: Descubrimiento causal

El Descubrimiento Causal (*causal discovery*) es el área de la inteligencia artificial y la estadística cuyo objetivo es inferir las relaciones de causa y efecto a partir de datos, en lugar de simplemente encontrar correlaciones. [2] Esto permite definir relaciones lógicas entre sucesos y puede traducirse al lenguaje de la lógica proposicional (dominio de SAT) con el concepto de la D-separación. [3] Cuando se trabaja con datos de muestras reales, las pruebas estadísticas de independencia suelen producir resultados contradictorios debido a la variabilidad estadística. Los métodos previos con frecuencia fallaban o se volvían ineficientes ante estas inconsistencias. [4] Mediante optimización de restricciones el problema de descubrimiento causal puede ser llevado al problema de MaxSAT. [4] Se propuso el uso de MaxSAT en [5] como el motor de búsqueda para el descubrimiento causal basado en restricciones.

Cada archivo del *benchmark* plantea una cantidad de casos para los cuales se aplica el descubrimiento causal y eso genera una cantidad de variables y cláusulas que llevan a los resolvers de MaxSAT a su límite. El resumen de los archivos se presenta en la tabla 1.

Tabla 1. Descripción de los archivos del *benchmark*

Archivo	Variables	Cláusulas	Casos	Lógica
n5	61600	221790	500 - 1k - 10k	UAI13
n6	328107	1206162	500 - 10k	UAI13
n6	12764	46236	500 - 1k - 10k	UAI14
n7	40290	145910	500 - 1k - 10k	UAI14

Solución exacta: EvalMaxSAT

EvalMaxSAT [6] es un solucionador de MaxSAT implementado en C++ y basado en el solucionador CaDiCaL [7] y el algoritmo aprendizaje un nivel a la vez (OLL por sus siglas en inglés: *One-Level-at-a-time Learning*) [8].

Solución heurística

Se propuso una heurística de ramificación estática (*Static Branching Heuristics*) basada en el conteo de literales. [9,10] La idea de la heurística se centra en tomar la variable cuya frecuencia de aparición sea máxima (regla de MOM [11]) para asignar el valor de verdad que maximice el número de cláusulas que se satisfacen en cada momento. La heurística es una simplificación de la heurística de Jerislow-Wang [12] que trabaja con esta misma idea pero con una función de costo afectada por el peso de cada cláusula.

Búsqueda local: vecindad 1-*flip*

La vecindad propuesta para cada solución consiste en alternar el valor de verdad asociado a una variable en una cláusula insatisfecha lo que implica que cada vecino de la solución difiere en exactamente una variable.

Búsqueda local iterada: perturbación y criterio de aceptación

La perturbación propuesta para esta metaheurística es la generalización *k-flip* sobre variables en cláusulas satisfechas. Esto incrementa la probabilidad de escape del cuenco de atracción que define la solución encontrada por la solución heurística propuesta al cambiar drásticamente la forma de la solución inicial. La búsqueda local se sigue haciendo con 1-*flip*.

Búsqueda tabú: movimientos tabú y criterio de aceptación

La lista tabú, donde se tienen los movimientos que no están permitidos, es un vector que guarda las iteraciones en que una variable no puede ser invertida, es decir, la variable en la posición *i*-ésima, va a ser tabú durante la cantidad de iteraciones que indique el valor del vector en el índice *i*.

Al comienzo, el costo de una solución se asume infinito y un movimiento se toma siempre y cuando mejore dicho costo (sea menor), por lo tanto, en la primera iteración siempre hay un movimiento a tomar. No se realizan los cambios en una iteración sino que se evalúa el costo de hacer un cambio y se elige el que tenga mayor impacto sobre la calidad de la solución si y solo si el movimiento no es tabú o es tan bueno que no es conveniente restringirlo.

La verificación de las iteraciones de restricción se verifica en $O(1)$ debido a que solamente se compara la iteración actual con el valor en un vector, sin que haya un recorrido. Dicha cantidad de iteraciones es variable con distribución uniforme. Se escoge un número entre 1 y 5 con probabilidad uniforme y eso garantiza que no se escoja restrinja siempre una variable una misma cantidad de iteraciones.

La calidad de la mejor solución global se almacena en todo momento y solamente se actualiza en caso de mejora estricta.

Recocido simulado: temperatura y enfriamiento

El enfriamiento implementado es geométrico, es decir, la temperatura decrece en un factor α cada iteración, por lo que, en la iteración *n*-ésima, se habrá enfriado en α^n . Antes de que se aplique el siguiente nivel de enfriamiento, se revisa por cierta cantidad de iteraciones, todos los vecinos posibles a esa temperatura, pero a diferencia de los vecinos explorados con la búsqueda tabú, aquí el vecino es seleccionado al azar. El criterio de aceptación viene por la probabilidad similar a la distribución de Boltzmann y si una solución candidata es mejor que lo que se tiene, se toma sin miramientos. Si no, la probabilidad de tomarla varía con la temperatura.

GRASP: la lista restringida de mejores candidatos

La función de costo consiste en seleccionar el máximo entre la cantidad de veces que una variable está negada o no (del mismo modo que en la solución heurística propuesta previamente). El umbral para entrar en la lista restringida de mejores candidatos se calcula con un factor α de 0,2. Esto satisface la condición voraz y aleatorizada.

La adaptatividad del algoritmo viene porque una vez seleccionada una parte de la solución, se actualizan las frecuencias asociadas a esa inclusión y entonces serán consideradas en la siguiente iteración.

Algoritmo genético

Para el algoritmo genético ya se tienen implementados el genotipo y el fenotipo de la población desde la solución heurística: el genotipo es la representación como vector de valores de las variables, donde el índice es la variable y el fenotipo es la representación global de la asignación como cadena binaria. La función de aptitud también está considerada y es justamente la cantidad de cláusulas insatisfechas.

Se implementaron los operadores de selección, cruce y mutación. El operador de selección de los padres es un torneo donde compiten tres candidatos de una generación. El operador de cruce es un cruce uniforme, es decir, para cada una de las n variables se escoge con igual probabilidad al padre A o al B y así se construye una nueva solución que no es demasiado parecida a ninguno de los dos padres. Luego, si hay n variables, la probabilidad de que una variable mute (invierta su valor) es $1/n$. Se probó con 50 individuos y 100 generaciones.

Resultados de corridas

En la tabla 2 se muestran los resultados en número de cláusulas insatisfechas promedio de dos ejecuciones de cada archivo del *benchmark*. El número entre paréntesis corresponde a la desviación estándar sobre la última cifra significativa. Si no tiene número entre paréntesis es porque la calidad de la solución no varió con las repeticiones.

Tabla 2. Calidad de las soluciones por algoritmo

Archivo	Casos	Exacto	Heurística	B. L.	B. L. I.	B. T.	R. S.	GRASP	A. G.
n5 i2	500	5	1952	222	191(1)	1512	1952		1887(3)
n5 i4	500	4	1949	219	184(4)	1509	1949		1914(4)
n5 i5	10k	5	1953	223	1,8(1)e2	1513	1953		1,84(6)e3
n5 i7	1k	10	1972	242	2,1(1)e2	1532	1972		1948(4)
n5 i8	10k	5	1977	247	217(4)	1537	1977		1946(8)
n6 i1	500		7672	623	575(9)	6402	7672		7647(3)
n6 i4	500		56	53	53(1)	51	56		56(1)
n6 i5	10k	4	7648	599	555(1)	6378	7648		7552(2)
n6 i7	1k	8	155	152	151	150	155		154
n6 i8	1k	8	144	141	140(1)	139	144		144
n6 i9	10k		69	66	65(1)	64	69		69(1)
n6 i9	1k		69	66	65	64	69		69
n7 i8	10k	20	378	372	372(1)	370	378		378
n7 i8	1k	20	378	372	371(1)	370	378		378(1)
n7 i9	500		220	214	214(1)	212	220		220

En la tabla 3 se puede observar el tiempo en segundos que le tomó a cada algoritmo con cada instancia del *benchmark*. El número entre paréntesis es la desviación estándar sobre la última cifra significativa.

Tabla 3. Tiempo de ejecución por algoritmo

Archivo	Casos	Exacto	Heurística	B. L.	B. L. I.	B. T.	R. S.	GRASP	A. G.
n5 i2	500	851.576	6(4)	10(3)	5(1)e2	2(1)	0.007(2)		17(1)
n5 i4	500	903.575	6(4)	11(3)	5(1)e2	1.5(5)	0.006(1)		17(3)
n5 i5	10k	1140.65	6(4)	9(3)	5(2)e2	1.7(5)	0.00796(3)		15.8(5)
n5 i7	1k	1941.66	6(4)	10(3)	5(1)e2	1.5(5)	0.006(1)		17(2)

Archivo	Casos	Exacto	Heurística	B. L.	B. L. I.	B. T.	R. S.	GRASP	A. G.
n5 i8	10k	143.131	6(4)	10(3)	4(1)e2	1.4(4)	0.0051(5)		17(2)
n6 i1	500		2(1)e2	2,6(9)e2	7(2)e3	4(1)	0.014(8)		6(1)e1
n6 i4	500		0,2(1)	0,002(1)	16(8)	0.3(1)	0.003(2)		2.5(9)
n6 i5	10k		2(1)e2	2,7(9)e2	7(2)e3	4(2)	0.016(6)		6(2)e1
n6 i7	1k	118.155	0,2(1)	0,003(2)	2(1)e1	0.3(2)	0.002(1)		2(1)
n6 i8	1k	124.095	0,2(1)	0,002(2)	2(1)e1	0.3(1)	0.0025(4)		2.5(9)
n6 i9	10k		0,2(1)	0,004(4)	2(1)e1	0.3(1)	0.002(1)		3(1)
n6 i9	1k		0,2(1)	0,003(2)	2(1)e1	0.2(2)	0.002(1)		2(1)
n7 i8	10k	253.064	3(2)	0,01(1)	3,1(3)	1.5(2)	0.005(1)		13(3)
n7 i8	1k	260.345	3(2)	0,01(1)	2,6(2)	1.1(5)	0.004(1)		12(6)
n7 i9	500		3(2)	0,02(2)	2,3(3)	1.1(5)	0.003(1)		10(5)

Se puede observar que la implementación de diferentes algoritmos lleva a distintos tiempos de ejecución, también la diferencia con los tiempos se debe a que la estructura de las cláusulas es muy densa [5] en estos problemas (cada cláusula puede tener suficientes variables para actuar como un cuello de botella). Asimismo, para esta entrega los distintos algoritmos fueron ejecutados en distintas máquinas lo que implica distinto poder de cómputo.

Referencias

- [1] da Silva, P. F. M. (2010). *Max-SAT algorithms for real world instances*. (Master Dissertation).
- [2] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.
- [3] Hyttinen, A., Hoyer, P. O., Eberhardt, F., & Jarvisalo, M. (2013). *Discovering cyclic causal models with latent variables: A general SAT-based procedure*. arXiv preprint arXiv:1309.6836.
- [4] Hyttinen, A., Eberhardt, F., & Jarvisalo, M. (2014, July). *Constraint-based Causal Discovery: Conflict Resolution with Answer Set Programming*. In UAI (pp. 340-349).
- [5] Berg, O. J., Hyttinen, A. J., & Jarvisalo, M. J. (2019). *Applications of MaxSAT in data analysis*. In International Conferences on Theory and Applications of Satisfiability Testing (pp. 50-64). EasyChair Publications.
- [6] Avellaneda, F. (2020). *A short description of the solver EvalMaxSAT*. MaxSAT Evaluation, 8, 364.
- [7] A. Biere, K. Fazekas, M. Fleury and M. Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020*.
- [8] A. Morgado, C. Dodaro, and J. Marques-Silva, *Core-guided MaxSAT with soft cardinality constraints*.
- [9] Silva, J. M., & Sakallah, K. A. (1996, November). *GRASP-a new search algorithm for satisfiability*. In Proceedings of International Conference on Computer Aided Design (pp. 220-227). IEEE.
- [10] Liang, J. H., Ganesh, V., Poupart, P., & Czarnecki, K. (2016, June). *Learning rate based branching heuristic for SAT solvers*. In International Conference on Theory and Applications of Satisfiability Testing (pp. 123-140). Cham: Springer International Publishing.
- [11] Dubois, O., André, P., Boufkhad, Y., & Carlier, J. (1996). *Sat versus unsat*. Johnson and Trick, Second DIMACS Series in Discrete Mathematics and Theoretical Computer Science, American Mathematical Society, 415-436.
- [12] Jeroslow, R. G., & Wang, J. (1990). *Solving propositional satisfiability problems*. Annals of mathematics and Artificial Intelligence, 1(1), 167-187.